

データ構造 永続データ構造 セグメント木 モノイド 区間クエリ

永続セグメント木

1. 概要

通常のセグメント木は、更新を行うたびに古い状態を破棄し、常に最新のバージョンだけを保持する構造になっています。それに対して **永続セグメント木** は、過去のすべてのバージョンに対してアクセスや更新が可能なセグメント木です。

永続セグメント木は、**永続データ構造** の入門として理解しやすいだけでなく、競技プログラミングにおいても幅広い応用が可能な強力なデータ構造です。本講座では、永続セグメント木の基本構造を解説し、その応用方法を実際の出題例をもとに紹介します。

2. 前提知識

本講座を理解するためには、**セグメント木** および **遅延セグメント木** に関する基礎的な知識が必要です。

3. 永続データ構造

通常、私たちが扱うデータ構造は **短命 (ephemeral)** です。つまり、データ構造に変更を加えた時に、古いバージョンを捨てて新しいバージョンだけを残します。それに対して、データ構造が **永続的 (persistent)** であるとは、データ構造に変更を加えた時に古いバージョンも残すことを言います。特に、過去の全てのバージョンへのアクセスと更新ができるデータ構造を **完全永続的 (fully persistent)** と呼びます。以降では永続と書いた場合は完全永続を意味します。

具体例を挙げて考えます。まず、次の基本的な問題を考えてみます。

【問題 1】

整数列 $A = (A_1, A_2, \dots, A_N)$ が与えられます。 Q 個のクエリが与えられるので処理してください。

q 個目のクエリは次の 2 種類の形式のいずれかです。

- $1 \text{ } i \text{ } x$: A の i 番目の要素の値を x に更新する。
- $2 \text{ } l \text{ } r$: A の l 番目から r 番目までの要素の最小値を出力する。

この問題はセグメント木を用いればクエリあたり $O(\log N)$ の時間計算量で解くことができます。では、次の問題はどうか？

【問題 2】

整数列 $A_0 = (A_{0,1}, A_{0,2}, \dots, A_{0,N})$ が与えられます。 Q 個のクエリが与えられるので処理してください。

q 個目のクエリは次の 2 種類の形式のいずれかです。

- $1 \text{ t } i \text{ } x$: 数列 A_t の i 番目の要素の値を x に更新してできる数列を A_q とする。
- $2 \text{ t } l \text{ } r$: A_t の l 番目から r 番目までの要素の最小値を出力する。

この問題も、クエリが **オフライン** である、すなわちあらかじめ Q 個のクエリが全て与えられる場合はクエリ先読みをした上でセグメント木を用いれば解くことができます。具体的には、更新クエリ ' $1 \text{ t } i \text{ } x$ ' を「 A_t から新しいバージョン A_q を作る操作」とみなすと、各バージョンの関係は木になるので、この木をセグメント木への 1 点更新を行いながら DFS することで各バージョンに対するクエリを処理できます。しかし、クエリが **オンライン** である、すなわちクエリに答えるたびに次のクエリが与えられる場合は通常のセグメント木では解くことができません。

また、平易な方法として「過去の全ての数列を配列に保存する」という方法も浮かびますが、これは当然クエリごとに時間空間ともに $O(N)$ の計算量が掛かり非常に非効率です。

この例のように、現在のデータ構造だけでなく過去のデータ構造の状態を効率よく活用したい時に永続データ構造は利用されます。（特に **【問題 2】** は永続データ構造を用いるとクエリあたり $O(\log N)$ の時間計算量で解くことができ、非常に効率的です。）

永続データ構造には様々な種類がありますが、今回は競技プログラミングで最も汎用性の高い **永続セグメント木** を解説します。

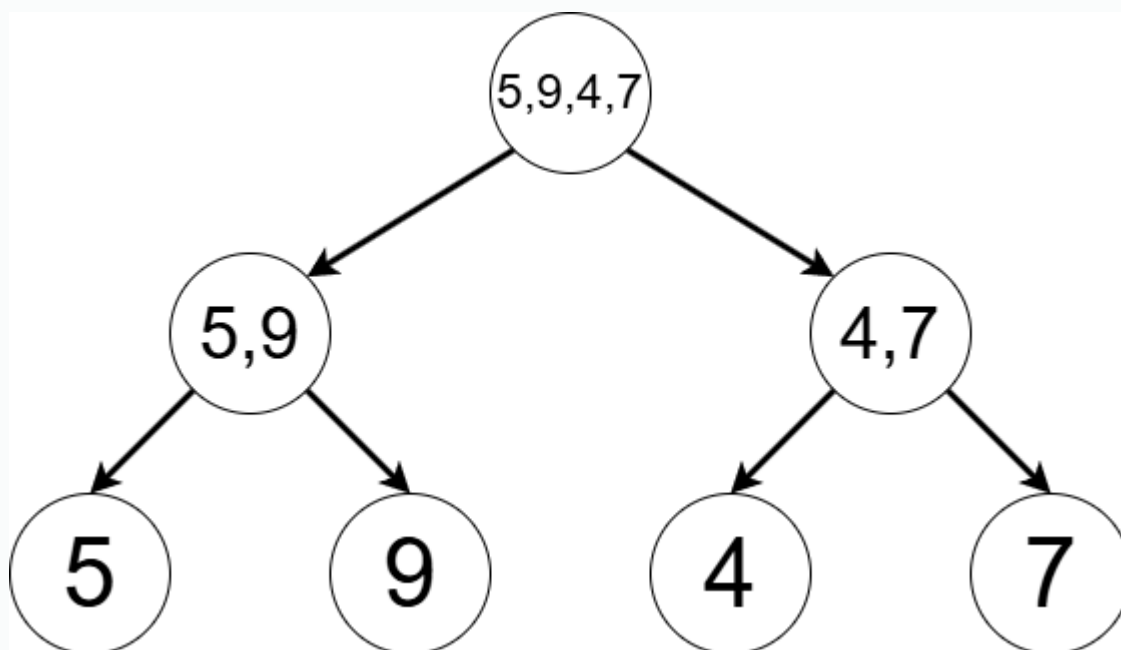
4. 永続セグメント木

永続セグメント木はセグメント木を永続化させたデータ構造です。本章では永続セグメント木の構造を説明します。

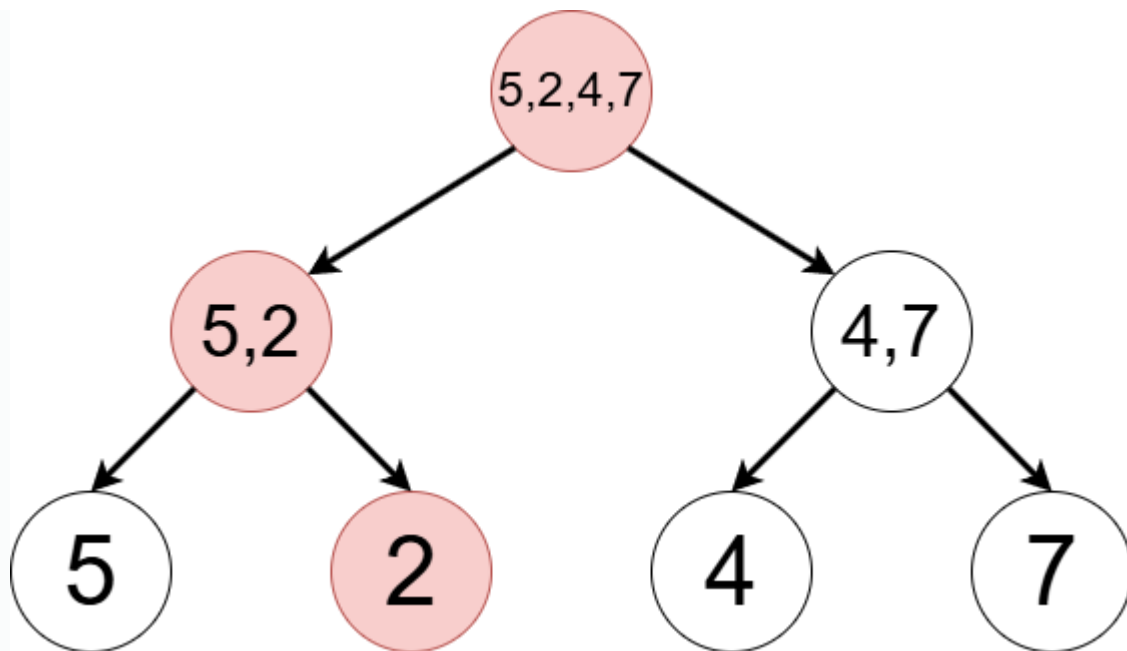
4.1. ポインタを利用したセグメント木

永続セグメント木はポインタを利用したセグメント木をベースとしたデータ構造です。（このようなセグメント木は **再帰セグメント木** と呼ばれることがあります。）永続セグメント木について理解するために、まずはポインタを用いたセグメント木の構造を振り返りましょう。

数列 $(5, 9, 4, 7)$ に対応するセグメント木の構造を図示したものが下図になります。セグメント木は **葉木** と呼ばれる構造をしています。すなわち、葉の頂点には数列の要素に対応する情報が載っていて、内部の頂点はそれらを 1 つにマージするための頂点となっています。内部の頂点は左右の子へのポインタをちょうど 2 個保持しています。



ここから 2 番目の要素を 2 に更新した場合、セグメント木の頂点の各状態は下の図に変化します。2 番目の要素に対応する葉、および祖先の頂点の値が書き換えられていることがわかります。書き換えが発生する頂点数は数列の長さを N として $O(\log N)$ 個です。また、木の構造、すなわち内部の頂点が示すポインタは変わらないこともわかります。



以上を踏まえた上で、永続セグメント木の構造を見ていきましょう。以下の 3 種類の操作を行う方法について解説します。

- 要素の更新
- 区間更新
- 同一区間のコピー

4.2. 要素の更新

まず、基本的なこととして、永続セグメント木の各バージョンをどのように管理するのかという問題があります。つまり、通常のセグメント木はバージョンが 1 個しかないので同定という概念がないのに対して、永続セグメント木は複数のバージョンが存在するため区別の必要があります。

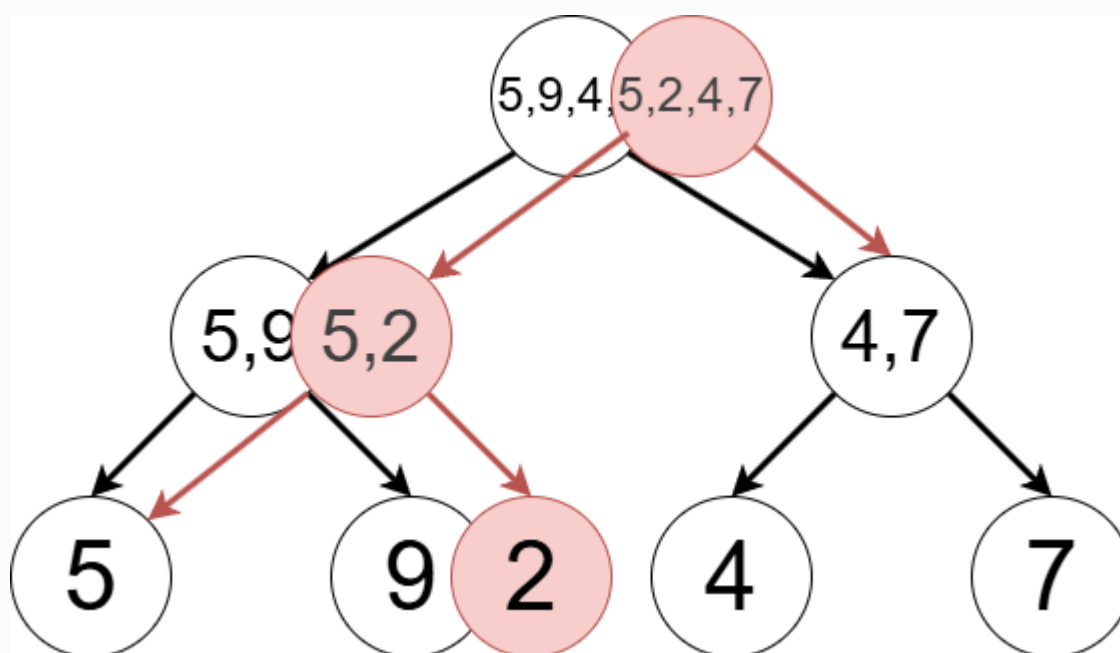
永続セグメント木は各バージョンに対して根を 1 対 1 に対応させることでこの問題を解決します。つまり、バージョンの異なる木同士の根は異なります。

これを踏まえた上で、永続セグメント木の要素を更新する方法について説明します。先ほど述べた通り、永続セグメント木はポインタを用いたセグメント木なので、数列の情報を保持する方法は通常のポインタを用いたセグメント木と全く一致します。

永続セグメント木に特有の操作が行われるのは要素を更新する場合です。先ほど述べた通り、通常のセグメント木では要素の更新が発生する頂点は $O(\log N)$ 個です。しかし、通常のセグメント木と同様に頂点の値を書き換えると、古いバージョンのセグメント木の情報が壊れてし

また、書き換えを行うことはできません。そこで、これらの $O(\log N)$ 個の頂点について、**頂点を複製して新たな頂点を作り、複製した頂点の情報を書き換える** というのが永続セグメント木の重要なポイントです。こうして頂点を複製した上で適切に辺を張ることで新たなセグメント木を作成して、その根を新しいバージョンとします。

下図は $(5, 9, 4, 7)$ という数列を管理する永続セグメント木から 2 番目の要素を 2 に変更して新しいバージョンのセグメント木を作る様子を図示したものです。(赤い頂点が複製された頂点) 観察すると、白い根頂点から探索を開始すると $(5, 9, 4, 7)$ に対応する数列が、赤い根頂点から探索を開始すると $(5, 2, 4, 7)$ に対応する数列が得られることがわかります。保持しているデータ全体はかなり複雑になりますが、特定の根から探索するという使い方をすると、通常のセグメント木と同様に扱えるという点がポイントです。このように頂点を複製して新しいセグメント木を作るという手法によって、古いバージョンと新しいバージョンのセグメント木が共存できていることがわかんと思います。



4.3. 区間更新

通常の 1 点更新のセグメント木だけでなく遅延セグメント木、すなわち区間更新のできるセグメント木を永続化することもできます。

遅延セグメント木の区間更新は以下の手順で行われます。

1. 根から葉の方向へ向けて、遅延評価用の値を伝播する。
2. $O(\log N)$ 個の区間に対応する頂点に作用素を作用させる。
3. 葉から根の方向へ向けて、作用素を作用させた後の値を計算していく。

この 3 個のステップで値が変化するのは $O(\log N)$ 個の頂点に限ります。よって通常の 1 点更新と同様に、情報を書き換える必要が発生するたびに新たな頂点を複製していく方式で新しいセグメント木を作成すればよいです。

例えば遅延評価用の値を伝播する際には、

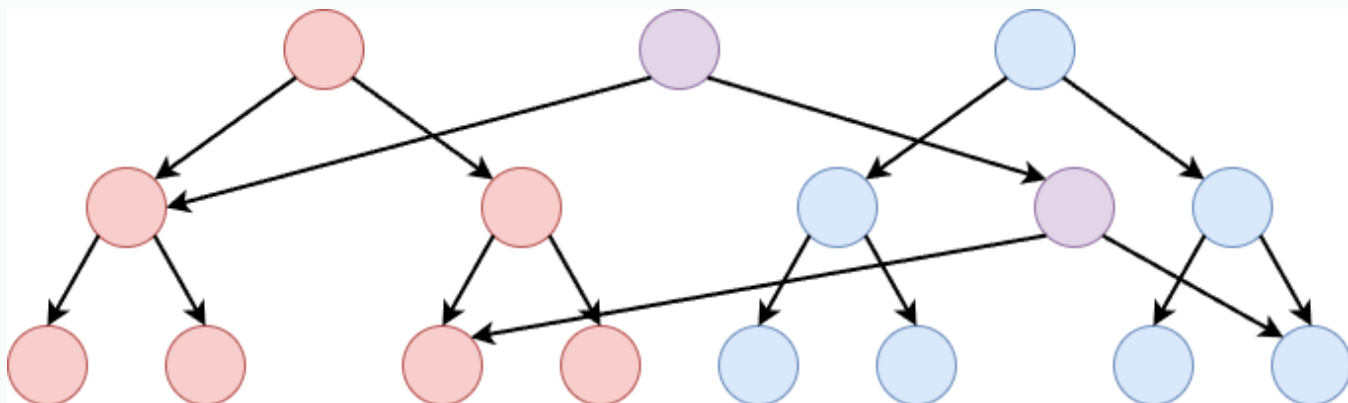
- 伝播させた後のノードに相当する自身のコピーを作る。
- 子ノードを複製し、複製してできた子にポインタを張り替える。これらのノードに遅延評価用の値を伝播させる。

という処理が必要になります。既存のノードに向かって伝播させることがないように注意してください。

4.4. 同一区間のコピー

通常のセグメント木ではできない特殊な操作として、同一区間のコピーが挙げられます。つまり、複数のバージョンの永続セグメント木があったときに、一方のバージョンのある区間の情報を他方のバージョンの同じ区間にコピーして新たなバージョンのセグメント木を作成することができます。セグメント木の区間は $O(\log N)$ 個の頂点で表現できるので、その部分の頂点を複製してポインタを張り替えればよいことがわかります。

例えば下図では、右のセグメント木に左のセグメント木の 1 番目から 3 番目の要素をコピーしてできるのが中央のセグメント木になります。



4.5. 実装例

永続セグメント木の要素の更新に関する実装例として以下に Library Checker の [Range Kth Smallest](#) への提出を挙げます。問題の解法に関しては第 5 章の [【問題 4】](#) で詳しく解

説明しているのでそちらを参照してください.

- <https://judge.yosupo.jp/submission/358107>

5. 問題例

競技プログラミングにおける永続セグメント木の使い方を説明していきます.

永続セグメント木は様々な問題に応用することができます. 先に挙げた **【問題 2】** のような明らかに永続データ構造を要求する問題だけでなく, 一見すると永続データ構造の出番がなさそうに見えて, 実はセグメント木を永続化させるとキレイに解くことができる問題が何問か存在します. そのような問題を紹介していきます.

【問題 3】

N 以下の正整数列 $A = (A_1, A_2, \dots, A_N)$ が与えられます. Q 個のクエリがオンラインで与えられるので処理してください.

各クエリでは l, r, x が与えられるので $(A_l, A_{l+1}, \dots, A_r)$ に含まれる x 以下の値の個数を求めてください.

前計算 $O(N \log N)$, クエリあたり $O(\log N)$ の時間計算量で解く方法を説明します.

Wavelet Matrix を利用した解法もありますが, ここでは永続セグメント木を用いた解法を説明します.

1 点更新・区間総和クエリを処理する永続セグメント木を考えます. S_0 を 0 埋めされた長さ N の列が載った永続セグメント木とします. そして, $i = 1, 2, \dots, N$ について S_{i-1} の A_i 番目の要素に 1 を加算したものを S_i とします.

すると, S_i の n 番目の要素は $(A_1, A_2, \dots, A_i)$ に含まれる n の個数になります. 直感的には, $A_1, A_2, \dots$ と順に値を追加していった時の頻度配列の変化が $S_1, S_2, \dots$ になっていると考えればよいです.

S_i を使用するとクエリへの回答は容易です. 求めたい答えは「 S_r の前 x 項の和」から「 S_{l-1} の前 x 項の和」を引いたものであるため, これは取得クエリで容易に計算できます.

【問題 4】

N 以下の正整数列 $A = (A_1, A_2, \dots, A_N)$ が与えられます. Q 個のクエリがオンラインで与えられるので処理してください.

各クエリでは l, r, k が与えられるので $(A_l, A_{l+1}, \dots, A_r)$ のうち小さい方から k 番目の要素を求めてください。

前計算 $O(N \log N)$, クエリあたり $O(\log N)$ の時間計算量で解く方法を説明します。

前計算する情報は先ほどと同じです。すなわち、永続セグメント木 $S_0, S_1, \dots, S_N$ であって S_i の n 番目の要素が $(A_1, A_2, \dots, A_i)$ に含まれる n の個数になるものを得られるように前計算を行います。

S_i を用いてクエリへの回答を行います。クエリで与えられる (l, r, k) について、求めたい答えは

- 「 S_r の前 i 項の和」－「 S_{l-1} の前 i 項の和」が k 以上となる最小の i

となります。このような i は「 i が条件を満たすか？」で二分探索を行い各二分探索内で取得クエリを呼ぶことで $O(\log^2 N)$ で求められますが、より高速な手法が存在します。

通常のセグメント木において、「 S の前 i 項の和が k 以上となる最小の i 」はセグメント木上の二分探索で $O(\log N)$ で計算できます。セグメント木上の二分探索の手法を観察すると、複数のセグメント木が同じ形状をしていればそれらの木を同時に下っていくことで二分探索ができることに気づきます。この手法によりクエリの答えを $O(\log N)$ で計算することができて、これは非常に高速です。

【問題 5】(UTPC 2011 問題 L : L 番目の数字)

出典

頂点に 1 から N の番号が書かれた N 頂点の木が与えられます。各頂点には数 a_i が対応づけられています。 Q 個のクエリを処理してください。

各クエリでは v, w, l が与えられるので、頂点 v と頂点 w を結ぶ単純パス上の頂点に対応付けられた数のうち l 番目に小さい数を求めてください。

UTPC 2011 というコンテストのボス問です。

【問題 4】 の手法を応用することで前計算 $O(N \log N)$, クエリあたり $O(\log N)$ の時間計算量で解くことができます。

まず、座標圧縮をすることで a_i は $1, 2, \dots, N$ のいずれかであると考えてよいです。木を頂点 1 を根とする根付き木とみなします。するとクエリは

- 「1 - v パス上の i 以下の数の個数」＋「1 - w パス上の i 以下の数の個数」－ $2 \times$ 「1 - $\text{lca}(v, w)$ パス上の i 以下の数の個数」＋「 $a_{\text{lca}(v, w)}$ が i 以下であれば 1, そうでなければ 0」

れば 0」が l 以上となる最小の i

が答えとなります。よって、各頂点 v について $1 - v$ パス上の数の頻度配列の区間和が計算できる永続セグメント木を保持すれば【問題 4】と同様の手法でクエリあたり $O(\log N)$ の時間計算量が達成できます。そして、各セグメント木は親のセグメント木に要素を追加する要領で $O(N \log N)$ の前計算で計算できます。以上よりこの問題を解くことができました。なお、想定解も「平衡二分木」を「セグメント木」と読み替えれば全く同じ手法を解説していることがわかります。(2011 年の日本ではセグメント木の概念が定着していなかったのでしょうか？興味深いです。)

【問題 6】(The 2nd Universal Cup. Stage 15, Problem C: Bladestorm)

出典

正整数 K が与えられます。

正整数列 B のコストを、次の操作を繰り返して B が空になるまでにかかる操作回数として定義します。

- $x = \max\{K, \min(B)\}$ とする。 B の全ての要素から x を引く。その後、0 以下の要素を全て取り除く。

$(1, 2, \dots, N)$ の順列 $A = (A_1, A_2, \dots, A_N)$ が与えられます。 $i = 1, 2, \dots, N$ について $(A_1, A_2, \dots, A_i)$ のコストを計算してください。

永続セグメント木とは関係ない見方をしていますが、実は永続セグメント木を用いれば $O(N \log N)$ の時間計算量で解くことができます。

まず、数列 B および正整数 S について次の問題の答えを $f(B, S)$ と置きます。

【問題 7】

数直線を考える。 B に含まれる各整数に対応する地点に黒い点を打つ。

そして、地点 S に立つ。その後、次の操作を繰り返す。

- 「右に K 移動」「右にある黒い点のうち最も近いものに移動」の 2 択のうち移動する距離が大きい方の操作を選び、移動する。

自身のいる地点より右に黒い点が存在しなくなったら操作を終了する。操作を行った回数
は？

このとき、問題の答えは $f((A_1, A_2, \dots, A_i), 0)$ になります。

そして、 $0 \leq s \leq N + K$ について $\text{dp}[s][i] := f((A_1, A_2, \dots, A_i), s)$ と置きます。求めたいものは $\text{dp}[0][*]$ です。

この $\text{dp}[s][*]$ を s の降順に計算していくことを考えます。 $s \geq N$ の場合は明らかに $\text{dp}[s][i] = 0$ です。 $s < N$ の場合に何が起きるかを考えると、

- $a_{s+1} = (A_t = s + 1 \text{ を満たす } t. \text{ ただしそのような } t \text{ が存在しない場合は } \infty)$
- $b_{s+1} = \min(a_{s+1}, a_{s+2}, \dots, a_{s+K})$

と a, b を置いた時に、

- $i \leq b_{s+1}$ の場合： $\text{dp}[s][i] = \text{dp}[s+1][i]$
- $i > b_{s+1}$ の場合： $\text{dp}[s][i] = \text{dp}[s+K][i] + 1$

が成り立つことが確認できます。よって $\text{dp}[s]$ は $\text{dp}[s+1]$ および $\text{dp}[s+K]$ に区間加算した列を適切に貼り合わせたものになるので、永続遅延セグメント木の区間のコピーを用いれば高速に計算できます。

他にも関連する問題を以下に挙げます。練習用にお使いください。

1. https://judge.yosupo.jp/problem/persistent_range_affine_range_sum
2. https://atcoder.jp/contests/abc453/tasks/abc453_g
3. <https://codeforces.com/contest/1771/problem/F>
4. https://atcoder.jp/contests/soundhound2018-summer-final/tasks/soundhound2018_summer_final_e

▶ 問題へのコメント

6. まとめ

本講座では、永続セグメント木の構造について解説しました。永続セグメント木は、永続データ構造の基本的な概念が凝縮された構造です。その仕組みを理解することで、永続配列や永続

平衡二分木といった他の永続データ構造もスムーズに学べるようになります。永続データ構造にはさまざまな種類が存在するため、興味がある方はぜひ他の分野にも触れてみてください。

また、問題例の章ではいくつかの具体的な応用例を扱いました。永続セグメント木は、複雑な区間クエリの処理や DP の高速化など、応用範囲の広いデータ構造です。これを自在に扱えるようになることで、解ける問題の幅が大きく広がります。ぜひ例題を実際に解きながら、永続セグメント木の考え方を身につけてください。

7. 参考文献

1. James R. Driscoll, Neil Sarnak, Daniel Dominic Sleator, Robert Endre Tarjan. Making Data Structures Persistent. Proceedings of the Eighteenth Annual ACM Symposium on Theory of Computing. pp. 109–121. 1986. DOI: 10.1145/12130.12142. <https://doi.org/10.1145/12130.12142>.
2. 37zigenのHP. 永続セグメント木・永続遅延セグメント木. URL: <https://37zigen.com/persistent-segment-tree/>. (2026-06-11 現在リンク切れ).
3. USACO Guide. Persistent Data Structures. <https://usaco.guide/adv/persistent?lang=cpp>. (閲覧日: 2026-04-05).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。 リアルタイムのオンラインコンテストで競い合うことや、 5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)